

结构体

数据类型

基本数据类型 `int\double\float\char`

复合数据类型 数组\指针\结构体

结构体是一种由用户自定义的新的类型，它可以把多个不同类型的成员打包在一起组合成一个新的类型

结构体类型定义

```
//定义结构体类型
struct student_s { // struct student_s 是结构体类型的名字
    int id;
    char name[25];
    char gender;
    float Chinese;
    float Math;
    float English;
}; //分号是不可省略的
```

结构体变量定义和初始化

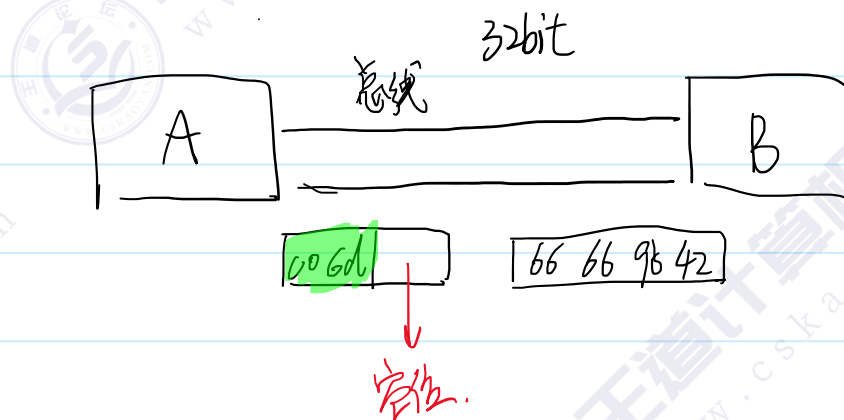
```
int main() {  
    //struct student_s stu; //结构体类型的变量  
    struct student_s stu = {1001, "Caixukun", 'm', 77.7, 66.6, 88.8};  
    return 0;    已用时间 <= 1ms  
}
```

sizeof 计算占据空间大小

&stu	0x000000
sizeof(stu)	44

e9 03 00 00
43 61 69 78
75 6b 75 6e
00 00 00 00
00 00 00 00
00 00 00 00
00 00 00 00
00 00 00 00
00 6d cc cc
66 66 9b 42
33 33 85 42
9a 99 b1 42

内存对齐



->运算符

```
struct student_s* pstu = &stu;  
// *运算符的优先级低于.运算符  
// ->运算符 先解引用再访问成员  
// (*pstu).id pstu->id  
printf("( *pstu).id = %d, pstu->id = %d\n", (*pstu).id, pstu->id);  
return 0;
```

Microsoft Visual Studio 调试控制台

```
stu = 1001 Caixukun m 77.699997 66.599998 88.800003  
stu1 = 1001 Caixukun m 77.699997 66.599998 88.800003  
(*pstu).id = 1001, pstu->id = 1001
```

typedef

给类型起别名

```
typedef int type_t; //给int起了别名叫type_t
//定义结构体类型
typedef struct { // struct student_s 是结构体类型的名字
    type_t id; //4+25+1+4+4+4
    char name[25];
    char gender;
    float Chinese;
    float Math;
    float English;
} student_t; //分号是不可省略的

//typedef struct student_s student_t;
int main() {
    //struct student_s stu; //结构体类型的变量
    student_t stu = {1001, "Caixukun", 'm', 77.7, 66.6, 88.8};
}
```

引用机制

C++的语法，简化指针的阅读难度

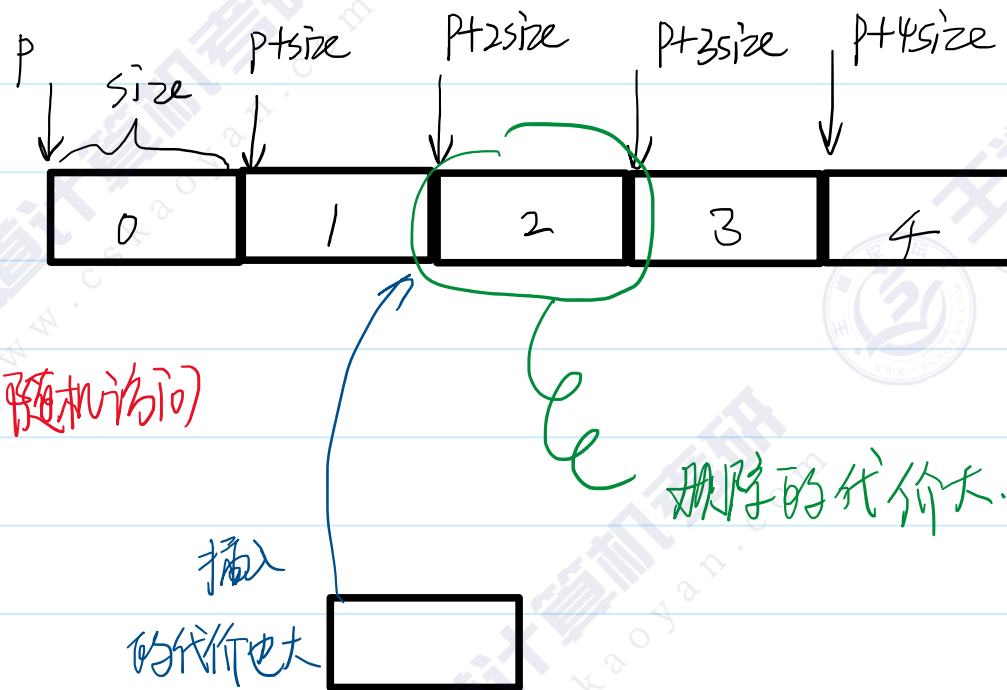
链表

数组vs链表

数组的特征

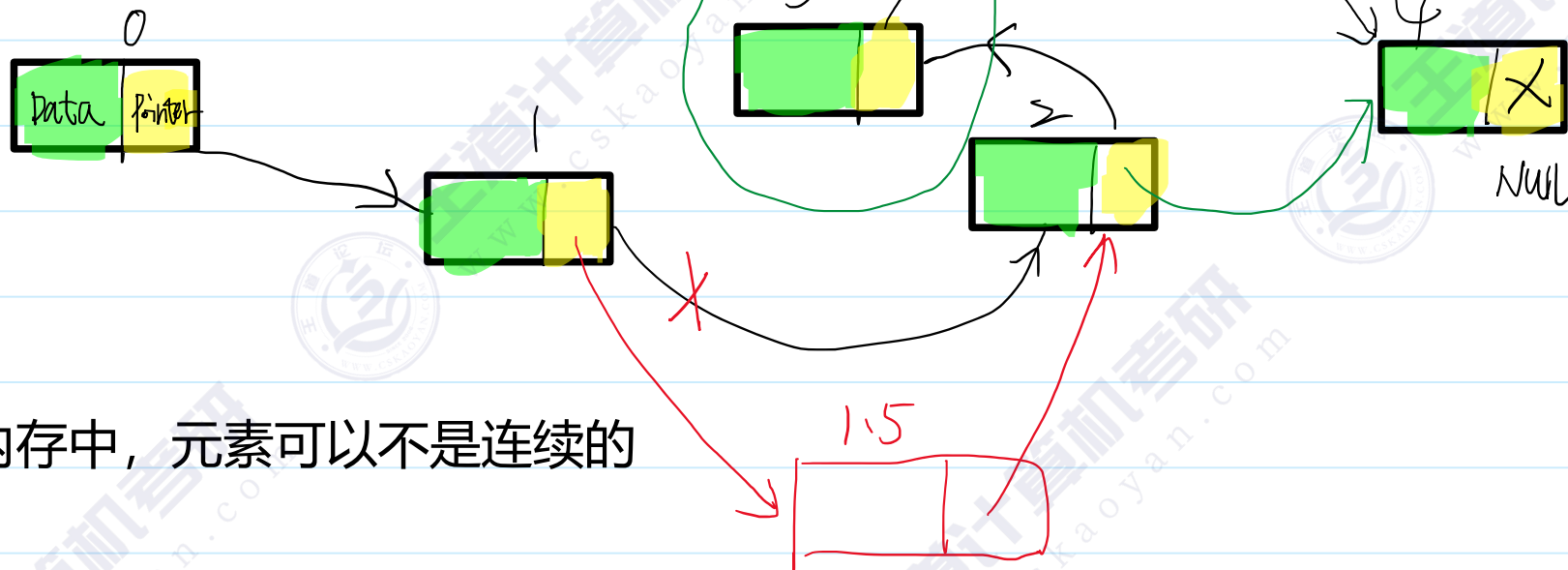
- 相同的元素类型
- 连续存放
- 有序的

→ 顺序线性表.



链表 也是一个线性表，它是一种链式结构，而不是顺序结构

链表的模型



在内存中，元素可以不是连续的

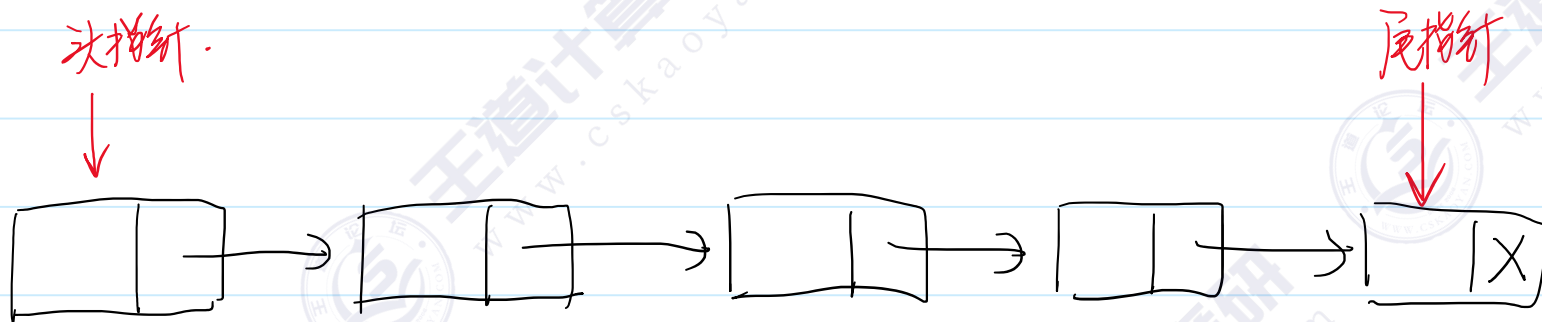
特征：

- 访问某个元素的效率低
- 空间利用率低

- 插入和删除的效率高

链表的代码实现

不带头结点 带头指针和尾指针 单链表



这种写法不对

```
struct node_s {  
    int data; //数据域  
    struct node_s next; //指针域 单链表只有一个指针域  
} node_t;
```

结构体在设计的时候需要能够知道它的大小 ---> 结构体定义的时候不能嵌套同名结构体 ---> 但是可以放指针

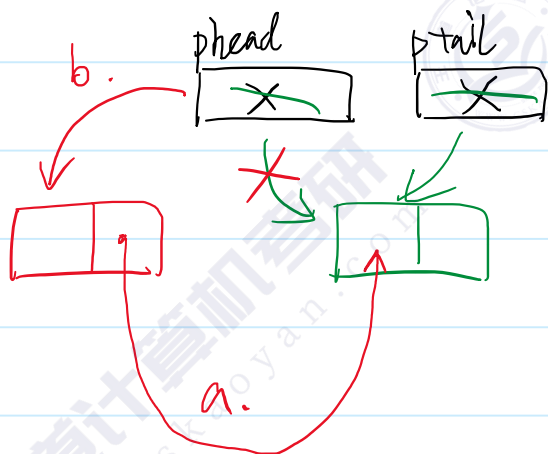
链表的设计

```
//先设计链表的某一个结点
typedef struct node_s {
    //数据域
    int data;
    //指针域
    struct node_s* next;
    //struct node_s next; //不行
} node_t;
typedef struct link_list_s {
    node_t* phead;
    node_t* ptail;
} link_list_t;
```

头插法

```
typedef struct link_list_s {  
    node_t* phead;  
    node_t* ptail;  
} link_list_t;
```

最开始链表为空 phead和ptail的指向都是NULL就可以了



- a. 将新结点的指针域指向原来的 phead.
- b. 将 phead 改成指向新结点.

phend



1 2

ptail



1 1

phend 63970
~~63BA0~~
0 - - - 0

ptail
~~63BA0~~
0 - - - 0

63BA0



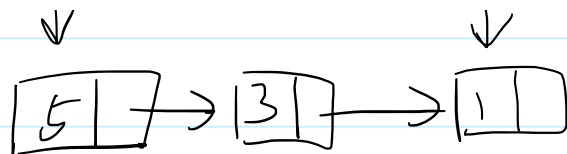
1	0...0
---	-------

63970

3	63BA0
---	-------

63910 63970

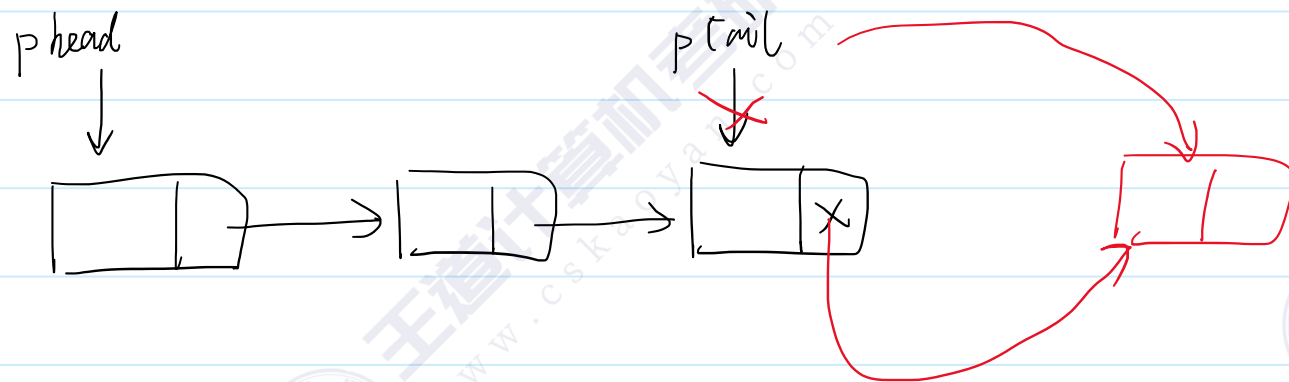
5	000
---	----------------



头插法代码

```
void head_insert(link_list_t* plist, int data) {  
    // 1 给新结点申请内存&&初始化  
    node_t* pnew_node = (node_t *)malloc(sizeof(node_t));  
    pnew_node->next = NULL; //新结点的指针域一开始总是NULL  
    pnew_node->data = data;  
  
    // 2 分类讨论  
    if (plist->phead == NULL) {  
        plist->phead = pnew_node;  
        plist->ptail = pnew_node;  
    }  
    else {  
        pnew_node->next = plist->phead;  
        plist->phead = pnew_node;  
    }  
}
```

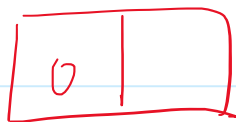
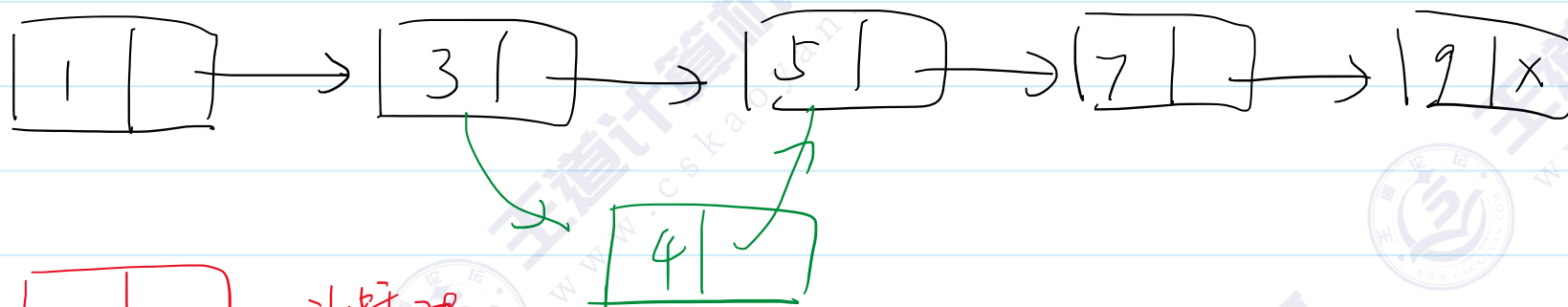
尾插法



尾插法代码

```
void tail_insert(link_list_t* plist, int data) {  
    // 1 给新结点申请内存&&初始化  
    node_t* pnew_node = (node_t*)malloc(sizeof(node_t));  
    pnew_node->next = NULL; //新结点的指针域一开始总是NULL  
    pnew_node->data = data;  
  
    // 2 分类讨论  
    if (plist->phead == NULL) {  
        plist->phead = pnew_node;  
        plist->ptail = pnew_node;  
    }  
    else {  
        plist->ptail->next = pnew_node;  
        plist->ptail = pnew_node;  
    }  
}
```

有序插入



头插法



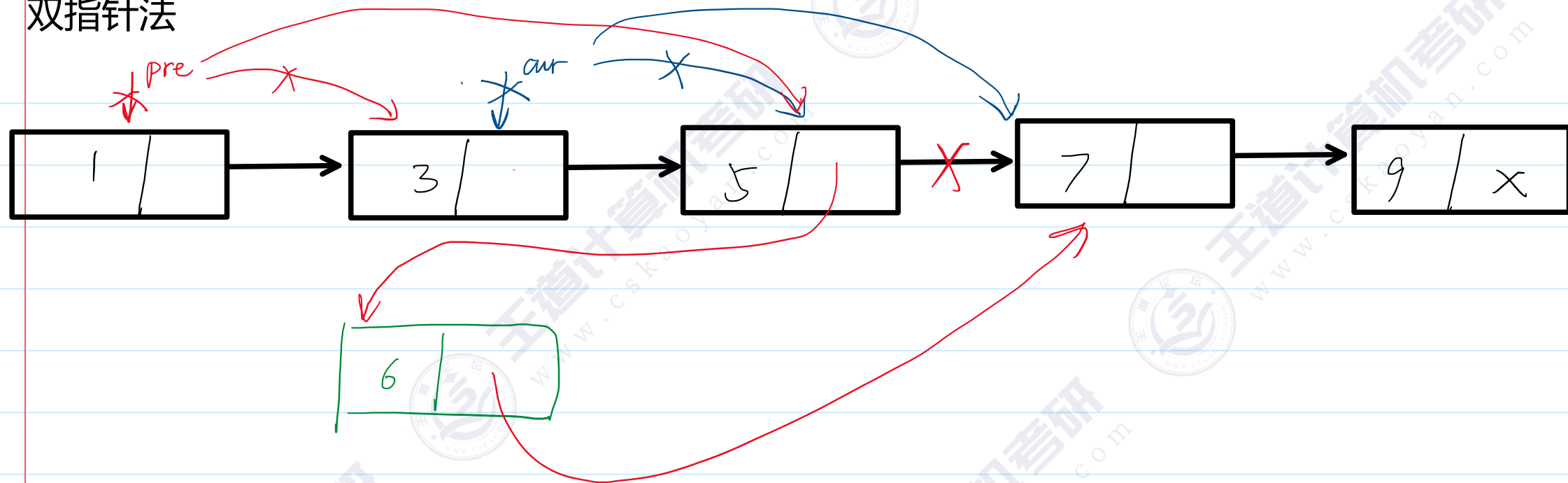
如果我们需要在一个结点的前面做插入，其关键是修改该结点前驱的指针域

双指针法：需要同时找两个指针

有序插入代码1

```
void sort_insert(link_list_t* plist, int data) {  
    // 1 给新结点申请内存&&初始化  
    node_t* pnew_node = (node_t*)malloc(sizeof(node_t));  
    pnew_node->next = NULL; //新结点的指针域一开始总是NULL  
    pnew_node->data = data;  
  
    // 2 分类讨论  
    if (plist->phead == NULL) { //没有结点的情况下不能用双指针  
        plist->phead = pnew_node;  
        plist->ptail = pnew_node;  
    }  
    else if (plist->phead->data > data) {  
        // 插入的结点比第一个结点都还要小, 退化成头插法  
        pnew_node->next = plist->phead;  
        plist->phead = pnew_node;  
    }  
}
```

双指针法



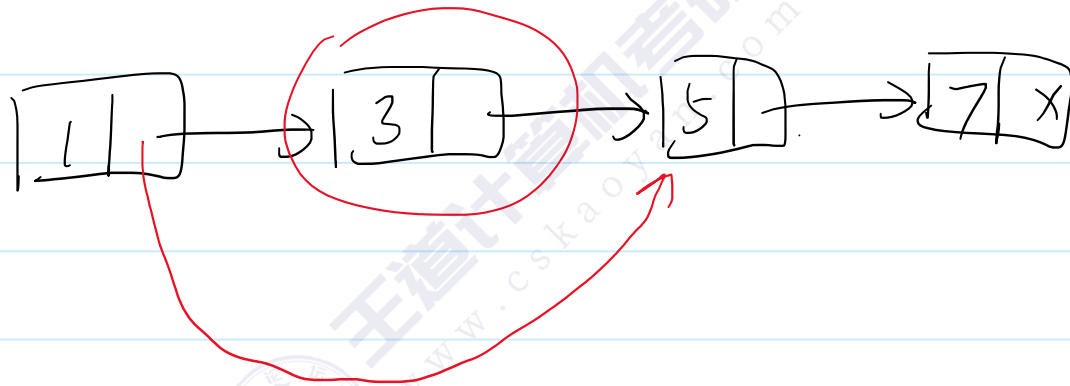
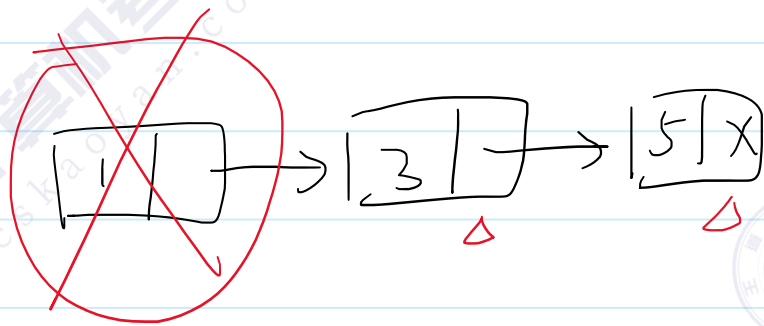
```
else { //使用双指针法寻找待插入的位置
    node_s* ppre = plist->phead;
    node_t* pcur = ppre->next;
    while (pcur != NULL) {
        if (pcur->data > data) {
            ppre->next = pnew_node;
            pnew_node->next = pcur;
            break;
        }
        ppre = ppre->next;
        pcur = pcur->next;
    }
    if (pcur == NULL) { //尾插法
        plist->ptail->next = pnew_node;
        plist->ptail = pnew_node;
    }
}
```

链表的删除

1、链表为空

2、删除链表第一个结点

3、删除其他结点



链表删除代码1

```
void list_delete(link_list_t* plist, int data) {  
    node_t* pcur = plist->phead; //pcur用来记录待删除结点的地址  
    if (plist->phead == NULL) {  
        printf("Error:list is empty!\n");  
        return;  
    }  
    else if (pcur->data == data) { //头部删除  
        plist->phead = pcur->next;  
        if (plist->phead == NULL) { //删除了之后，链表为空  
            plist->ptail = NULL;  
        }  
    }  
}
```

链表删除代码2

```
,  
else { // 某一个结点的指针域会发生改变, 用双指针法  
    node_t* ppre = plist->phead;  
    pcur = ppre->next;  
    while (pcur != NULL) {  
        if (pcur->data == data) {  
            ppre->next = pcur->next;  
            break;  
        }  
        ppre = ppre->next;  
        pcur = pcur->next;  
    }  
    if (pcur == NULL) { //待删除的结点不存在, 报错  
        printf("Error:no such node!\n");  
        return;  
    }  
    if (pcur == plist->ptail) {  
        plist->ptail = ppre;  
    }  
}  
  
free(pcur);  
pcur = NULL;
```